

latex2plain

Convert LaTeX math notation in Markdown files to clean, readable plain Unicode text.

```
f(n) \leq C \cdot g(n) \quad \text{for all } n \geq n_0
```

becomes:

```
f(n) ≤ C · g(n)    for all  n ≥ n0
```

latex2plain runs as both a **CLI tool** and a **web application** with a React frontend and Express backend.

Table of Contents

- [CLI Installation & Usage](#)
 - [Web Application](#)
 - [Conversion Pipeline](#)
 - [Symbol Reference](#)
 - [Local Development](#)
 - [Render Deployment](#)
 - [Environment Variables](#)
 - [Architecture](#)
 - [License](#)
-

CLI Installation & Usage

Install

```
npm install -g latex2plain
```

Or run locally after cloning:

```
npm install
npm run build
npm install -g .
```

Usage

```
latex2plain [options] [input] [output]
latex2plain [options] <directory>
cat notes.md | latex2plain
```

Examples

```
# Overwrite file in place
latex2plain notes.md

# Write to a new (plain Markdown) file
latex2plain notes.md clean.md

# Generate an accessible HTML page from the converted output
latex2plain notes.md clean.html

# Generate a PDF from the converted output
latex2plain notes.md clean.pdf

# Preview without writing
latex2plain notes.md --dry-run

# Show what changed
latex2plain notes.md --diff

# Read from stdin, write to stdout
cat notes.md | latex2plain

# Smart-format raw study notes before conversion
latex2plain notes.md --smart-format clean.md

# Convert all .md files in a directory (recursive)
latex2plain docs/
```

Options

Flag	Description
<code>-h, --help</code>	Show help
<code>-v, --version</code>	Show version
<code>-y, --yes</code>	Skip overwrite confirmation
<code>--dry-run</code>	Preview output without writing
<code>--diff</code>	Show before/after diff
<code>--stdin</code>	Force reading from stdin
<code>--verbose</code>	Print conversion stages
<code>--no-subscripts</code>	Skip subscript conversion

<code>--no-superscripts</code>	Skip superscript conversion
<code>--no-fractions</code>	Skip fraction conversion
<code>--no-roots</code>	Skip root conversion
<code>--no-mappings</code>	Skip symbol mapping
<code>--no-stripping</code>	Skip stripping wrapper commands (font, color, accents, boxes, etc.)
<code>--no-environments</code>	Skip environment stripping (<code>\begin{...}</code> / <code>\end{...}</code>)
<code>-s, --smart-format</code>	Auto-format raw text into structured Markdown before conversion

Web Application

latex2plain includes a full web application where users can convert Markdown and LaTeX to PDF without installing anything.

Features

- **Paste text** — Type or paste Markdown/LaTeX directly into the editor
- **Upload files** — Upload `.md` or `.txt` files (drag & drop supported)
- **Smart Format toggle** — Auto-detect headings, code blocks, math formulas, definitions, and more from raw unstructured text before conversion
- **AI Format toggle** — Uses Groq AI (Llama 3) to restructure messy/unformatted notes into beautiful Markdown before conversion. Falls back to Smart Format when AI is unavailable or rate-limited.
- **3 conversion modes:** Raw (passthrough), Smart Format (deterministic), AI Format (AI-powered)
- **Real-time progress stepper** — Visual timeline showing each conversion step (sending → processing → formatting → PDF generation)
- **Unrecognized LaTeX detection** — Remaining `\command{...}` patterns get red error boxes in the PDF with the salvaged content visible
- **Generate PDF** — Click "Convert to PDF" to download the result
- **Cold-start UX** — Time-based loading messages inform users when the Render backend is waking up from inactivity
- **Responsive design** — Works on mobile, tablet, and desktop
- **Dark mode** — Automatically respects system `prefers-color-scheme`

Try It

Visit the live web app (if deployed) or run locally (see [Local Development](#)).

Conversion Pipeline

Smart Format (pre-processing)

The optional **Smart Format** stage runs before the main conversion pipeline when `--smart-format` is enabled. It analyzes raw unstructured text and produces structured Markdown:

Detection	Raw Input	Formatted Output
Headings	chapter 1: calculus review	# chapter 1: calculus review
Sub-headings	2.1 Methodology	### Methodology
Code blocks	def factorial(n): / import numpy	@@CODEBLOCK-6@@
Math expressions	int_a^b f(x) dx	\$\$int_a^b f(x) dx\$\$
Definitions	def: A function maps...	**Definition:** A function maps...
Notes/Callouts	note: Remember to check...	> **Note:** Remember to check...
Examples	example: Compute the derivative	**Example:** Compute the derivative
Formulas	formula: E = mc^2	\$\$E = mc^2\$\$
Implicit lists	Logistic regression - 87.3%	- Logistic regression – 87.3%
Bare URLs	https://example.com	example.com

This is especially useful for:

- Raw study notes from lectures or textbooks
- ChatGPT output with code, math, and explanations mixed together
- Research paper drafts with section titles but no markdown formatting
- Any unstructured text with embedded code, math, or structured content

Main Pipeline

Stage	Input	Output
Code block protection	@@CODEBLOCK-7@@	placeholder (restored after)
Dollar stripping	x^2 → x^2 , $\$ \$ \backslash n \dots \backslash n \$ \$$ → ...	content with delimiters removed
Display math stripping	x^2	x^2
Style command stripping	$$	(removed)

<code>\pmod{}</code>	<code>\pmod{n}</code>	$(\text{mod } n)$
<code>\lim</code> formatting	<code>\lim_{x \to \infty}</code>	$\lim (x \rightarrow \infty)$
Symbol mappings	<code>\alpha</code> → α , <code>\infty</code> → ∞ , <code>\rightarrow</code> → $\rightarrow$	Unicode symbols
<code>\text{}</code>	<code>\text{hello}</code>	hello
<code>\tag{}</code>	<code>\tag{1}</code>	(1)
Font commands	<code>\mathrm{xyz}</code> , <code>\mathbf{x}</code> , etc.	inner content only
Color commands	<code>\color{red}{x}</code> , <code>\textcolor{red}{x}</code>	inner content only
Cancel, sout, underline, etc.	<code>\cancel{x}</code> , <code>\sout{x}</code>	inner content only
Boxes	<code>\boxed{x}</code> , <code>\fbox{x}</code>	x
Accents	<code>\vec{x}</code> , <code>\hat{x}</code> , <code>\bar{x}</code> , etc.	inner content only
Environments	<code>\begin{align}...\end{align}</code>	inner content with <code>\\</code> → ;
<code>\binom{}{}</code>	<code>\binom{n}{k}</code>	$C(n,k)$
Subscripts	<code>n_0</code> , <code>x_{123}</code>	n_0 , x_{123}
Superscripts	<code>x^2</code> , <code>e^{-n}</code>	x^2 , e^{-n}
Fractions	<code>\frac{1}{2}</code>	$(1/2)$
Roots	<code>\sqrt{x}</code> , <code>\sqrt[3]{x}</code>	$\sqrt{x}$, $\sqrt[3]{x}$
Function parenthesization	<code>\sin x</code> → <code>\sin(x)</code> , <code>\log n</code> → <code>\log(n)</code>	parentheses added (step 20.5, before fractions to avoid double-wrap)
Cleanup	extra spaces / blank lines	normalized
Code block restoration	placeholder	original <code>```</code>

See the [full stripping reference](#) for all supported commands.

Local Development

Prerequisites

- Node.js 20+
- npm 9+

Setup

```
# Clone the repository
git clone https://github.com/muddassir-2025/latex2plain-md-tool.git
cd latex2plain

# Install server dependencies
npm install

# Install client dependencies
cd client && npm install && cd ..

# Build everything
npm run build:all
```

Run Commands

```
# Run the CLI
npm run start -- input.md output.pdf

# Run the backend server (serves API on :3001)
npm run server

# Run the React frontend dev server (with Vite proxy to backend)
npm run dev:client

# Run both backend + frontend in development
npm run dev:all

# Run tests (431 tests across 17 test files)
npm test

# Run lint
npm run lint
```

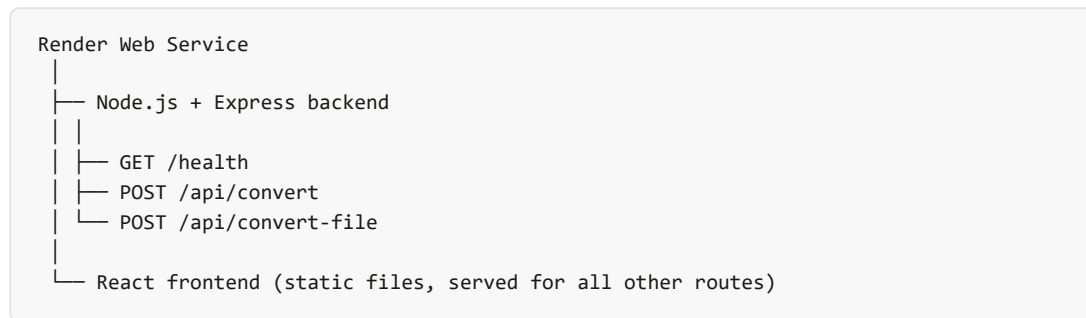
Development Flow

1. Start the backend: `npm run dev:server`
 2. In another terminal, start the frontend: `npm run dev:client`
 3. Open <http://localhost:5173> in your browser
 4. The Vite dev server proxies `/api/*` requests to the Express backend on port 3001
-

Render Deployment

Architecture

The application is designed as a single **Render Web Service** that serves both the API and the React frontend:



Deploy Steps

1. **Push to GitHub** (or connect your repository to Render).
2. **Create a Web Service** on Render:
 - **Branch:** `main`
 - **Runtime:** `Docker` (uses the included `Dockerfile`)
 - **Build Command:** (leave empty — the Dockerfile handles building)
 - **Start Command:** (leave empty — the Dockerfile handles starting)
 - **Environment Variables:**
 - `NODE_ENV` : `production`
 - `PORT` : (Render sets this automatically)

Docker Deployment

The included `Dockerfile` uses a multi-stage build:

1. **Build stage:** Compiles TypeScript and builds the React frontend
2. **Production stage:** Installs Chromium system dependencies for Puppeteer and starts the server

The Dockerfile sets `PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium` so Puppeteer uses the system-installed Chromium rather than downloading its own.

Render Free Tier

- **Cold starts:** After 15 minutes of inactivity, Render's free web services spin down. The first request will take longer (typically 30–60 seconds).
- **Cold-start UX:** The React frontend displays time-based messages:
 - 0–3 seconds: "Preparing your PDF..."
 - 3–8 seconds: "The server may need a moment to start..."
 - 8+ seconds: "Still working... Your PDF is being generated."
- **Bandwidth:** Free tier includes 500 MB outbound bandwidth per month.
- **CPU/Memory:** Free services have limited resources; large documents may take longer.

Puppeteer/Chromium Notes

- The Dockerfile installs Chromium system-wide and sets `PUPPETEER_EXECUTABLE_PATH`.
- PDF generation uses `md-to-pdf` which wraps Puppeteer under the hood.
- If deploying without Docker, you may need to configure Puppeteer to find Chromium on Render's base Node image.

Environment Variables

Variable	Default	Description
<code>PORT</code>	<code>3001</code>	Port for the Express backend (Render sets this automatically)
<code>NODE_ENV</code>	<code>development</code>	Set to <code>production</code> for Render deployment
<code>VITE_API_URL</code>	(empty)	API URL for the React frontend in production. In development, Vite proxies requests automatically. Set to your Render URL in production (e.g. <code>https://your-app.onrender.com</code>)

Copy `.env.example` to `.env` and fill in the values:

```
cp .env.example .env
```

Architecture

Raw unstructured text (study notes, ChatGPT output, ...)

|

▼

Smart Format (src/formatter/)
(7-pass detection & structuring)

|

▼

latex2plain core (src/converter/)
(LaTeX → Unicode pipeline)

|

|

▼

▼

CLI (src/cli.ts) Web API (src/server.ts)

|

▼

▼

Terminal users React frontend (client/)

|

▼

Browser users

Core Modules

Module	Location	Purpose
Conversion pipeline	src/converter/	LaTeX → Unicode text conversion (27 stages)
Smart Format	src/formatter/	Pre-processes raw text into structured Markdown (7-pass detection)
PDF generation	src/pdf.ts	Wraps md-to-pdf for reusable PDF buffer output
CLI	src/cli.ts	Command-line interface, flag parsing, file processing
Web server	src/server.ts	Node.js HTTP server with Express API for convert/health endpoints
HTML template	src/html.ts	Accessible HTML5 page generation
PDF styles	src/pdf-style.ts	CSS for md-to-pdf output
React frontend	client/	Vite + React SPA with paste/upload UI

Reusable Core

The key design principle is that the **conversion engine is shared** between the CLI and web API:

- **CLI:** Calls `convert()` directly, then writes to files or uses `mdToPdf` for PDF output

- **Web API:** Calls `convertToPdfBuffer()` which internally calls `convert()` + `mdToPdf`, returns a Buffer

Both reuse the same pipeline — no shell commands, no child processes, no code duplication.

Tests

```
npm test # Run all 431 tests (17 test files)
npm run test:watch # Watch mode
```

Test files cover:

- All conversion pipeline stages (12 test files, 250+ tests)
 - Smart Format formatting (77 tests)
 - AI Format integration (26 tests)
 - LaTeX stripping (72 tests)
 - Unrecognized LaTeX detection (21 tests)
 - Metadata and structure preservation (49 tests)
 - Server API endpoints (8 tests)
 - Input validation
 - Empty inputs
 - Error handling
-

License

MIT